

Article title

TinZr: A Compact Wireless ESP32-C3 Platform for Multi-modal Physiological Data Acquisition.

Authors

Ludvik Alkhoury^{1,*}, Tony Moore², Petras Swissler³, N. Jeremy Hill^{4,5}, Sudhin A. Shah¹, and Moshe Kam^{6,*}

Affiliations

¹ Department of Radiology, Weill Cornell Medicine, New York, NY, 10065 USA

² Mortronics Limited, Folkstown, Balbriggan, Dublin, Ireland

³ Mechanical Engineering Department, New Jersey Institute of Technology, Newark, NJ, USA

⁴ National Center for Adaptive Neurotechnologies, Stratton VA Medical Center, Albany, NY, USA

⁵ Electrical and Computer Engineering Department, State University of New York at Albany, NY, USA

⁶ Electrical and Computer Engineering Department, New Jersey Institute of Technology, Newark, NJ, USA

Corresponding authors email addresses

*Ludvik.alkhoury@gmail.com and Kam@njit.edu

Abstract

Microcontrollers have become central to wearable technologies. As these systems continue to evolve, both form factor and functional integration have become critical design considerations. In this study, we present *TinZr*, a compact, wireless, and open-source controller based on the ESP32-C3 microcontroller. The *TinZr* integrates an inertial measurement unit (IMU), an onboard battery management circuit, Wi-Fi and Bluetooth Low Energy, a microSD card reader, and multiple GPIOs supporting digital, analog, I²C, SPI, and UART interfaces. The complete system measures only 25 mm × 25 mm, making it well-suited for wearable applications. All production files, including PCB design, firmware, and a 3D model of a wearable enclosure, are publicly available. Two primary use cases for *TinZr* are demonstrated: (a) multi-device communication and coordination, and (b) wireless IMU data acquisition. The *TinZr* supports both real-time data streaming via Wi-Fi/BLE and local logging to a microSD card. Local streaming makes the *TinZr* suitable for extended research experiments and field studies where continuous wireless connectivity may not be available. An accompanying Arduino library, *TinZrConnect*, is provided to facilitate programming and integration with existing systems. Overall, the combination of compact design, wireless flexibility, and open-source accessibility, makes *TinZr* a versatile solution for various sensing applications.

Keywords

Microcontroller, ESP32-C3, Wearable Devices, Physiological Signals, Physiological Monitoring, Data Acquisition, Multimodal Signals, Wireless Communication, Multi-Device Synchronization, Open-Source Hardware, Signal Processing, Inertial Measurement Unit (IMU), Low-Power Design, MicroSD Data Logging, Battery-Powered Systems

1. Specifications table

Hardware name	TinZr
Subject area	• Electrical and biomedical engineering • Neuroscience • Swarm robotics • Educational tools and open source alternatives to existing infrastructure
Hardware type	• <i>Measuring physical properties and in-lab sensors</i> • <i>Field measurements and sensors</i> • <i>Electrical and biomedical engineering</i> • <i>Multi-functional microcontroller platform</i> • <i>Compact and wearable embedded controller</i> • <i>Wireless data acquisition and communication system (Wi-Fi / BLE)</i> • <i>Real-time signal processing</i> • <i>Physiological signal acquisition and sensor interfacing (ECG, PPG, IMU)</i> • <i>Supports digital communication protocols (I²C, SPI, UART, GPIO)</i>
Closest commercial analog	• Seeed XIAO ESP32-C3 • ESP32-C3 Mini • DFRobot Beetle ESP32-C6 • Adafruit GEMMA v2 • Teensy 4.1
Open source license	CERN Open Hardware Licence (OHL) v2.0 Permissive
Cost of hardware	\$65 (including shipping and customs duties)
Source file repository	https://osf.io/hx4cm/
OSHWA certification UID	US002796

2. Hardware in context

Wearable sensing platforms are gaining popularity for their ability to deliver non-invasive, continuous health insights to athletes, patients, and everyday users – while evolving to incorporate sophisticated on-device sensors and analytics. The global wearable sensors market was valued in 2024 at USD 1.4 billion and is projected to grow to USD 6.3 billion by 2033, at a compound annual growth rate (CAGR) of 18.3% over 2025–2033 [1, 2]. This surge is largely driven by growing demand for telehealth, long-term monitoring, real-time health tracking, and military applications.

Physiological signal monitoring has become increasingly widespread, with wearable systems now employed by athletes, patients, soldiers, and everyday users to enable continuous health and performance tracking [3, 4, 5, 6]. Consequently, many everyday wearables (*e.g.*, watches) now go beyond basic functionality by including onboard sensing hardware, such as accelerometers for activity and motion tracking [7]; photoplethysmography (PPG) sensors for heart rate [8] and oxygen saturation calculation [9]; electrocardiography (ECG) sensors for cardiac rhythm monitoring [10]; temperature sensors for body or skin thermal profiling [11]; and even biochemical sensors for measuring variables like glucose [12], for richer health and activity monitoring [4, 13].

Wearable systems for physiological monitoring increasingly depend on microcontroller units (MCU) and platforms that provide tight integration, compact form factors, and support for multi-modal sensing. The continued miniaturization and increasing computational capability of modern MCUs have made it possible to embed real-time sensing, processing, and wireless communication within small, battery-powered enclosures. Additionally, such MCUs offer sufficient local processing power, robust data logging, and versatile interfaces for auxiliary modules such as inertial, optical, and electrophysiological sensors.

Several open hardware and research platforms have emerged to address these challenges. For instance, *CogWatch* [14] is an open-source multi-sensor wrist platform that integrates heart rate, respiration, skin conductance, and temperature sensing, to study physiological and behavioral responses in daily activities. *CogWatch* uses the *Adafruit QT Py ESP32-S2* MCU [15] for data acquisition and measures around 29 mm x 38 mm (just the PCB without any sensor enclosures). *H-Watch* [16] extends these capabilities into a wristwatch-like platform that incorporates multiple radios and energy harvesting, maintaining ultra-low power consumption for long-term wearable deployment. The main purpose of *H-Watch* is early COVID-19 detection. *BioGAP-Ultra* [17] is a modular edge-AI platform that supports synchronized acquisition of EEG, EMG, ECG, and PPG signals with on-device processing and flexible form factors. The *BioGAP-Ultra* firmware runs on the nRF5340 SoC using the Zephyr RTOS, and the accompanying Android software suite enables background data reception, real-time visualization, some parameter configuration, and data storage. Finally, the *Multi-Sensor Wearable Health Device (MWHD)* framework [18] uses the *CC2640R2F Cortex@-M3* [19] MCU for real-time heart rate and peripheral oxygen saturation (SpO₂) monitoring from PPG signals, temperature, and motion, coupled with wireless data transmission and mobile application feedback. Together, these platforms illustrate the state-of-the-art in wearable microcontroller integration, where the convergence of low-power embedded processing, miniaturization, and open hardware design has enabled compact, reconfigurable, and affordable solutions for physiological signal monitoring.

Some of the above-mentioned platforms demand significant expertise to program or extend at the driver and firmware level. However, there are compact and flexible MCUs that can be easily programmed via the Arduino ecosystem – making them highly accessible for rapid prototyping, research deployments, and wearable device development. For example, we list several widely used controllers such as *Seeed XIAO ESP32-C3* [20], *DORHEA ESP32-C3 Mini* [21], *DFRobot Beetle ESP32-C6* [22], *Adafruit GEMMA v2* [23], and *PJRC Teensy 4.1* [24]. In general, most of these controllers are compact and thus suitable for wearable applications. The *Seeed XIAO ESP32-C3* is very compact and offers Wi-Fi/BLE connectivity with onboard battery management, though it lacks a microSD card reader and has limited header accessibility. The *DORHEA ESP32-C3 Mini* does not include battery management or an SD card interface. The *DFRobot Beetle ESP32-C6* similarly lacks an SD card reader and accessible headers. The *Adafruit GEMMA v2* is extremely small and lightweight but does not include a charging unit, SD card reader, and expansion headers. The *PJRC Teensy 4.1* includes an SD card reader, making it well-suited for data logging, but it lacks Wi-Fi/BLE and is physically bulkier than the other options.

In this study, we introduce the *TinZr*, which builds on existing designs by providing a fully wireless con-

troller in a compact 25 mm × 25 mm form factor, combining sensing, data logging, and communication in a minimal open-source package. It uses the ESP32-C3 MCU and integrates an inertial measurement unit (IMU) comprising a 3-axis accelerometer and 3-axis gyroscope, battery management circuitry, Wi-Fi and BLE 5.0 connectivity, a microSD card reader, and GPIOs supporting digital, analog, I²C (it provides an on-board QWIIC connector), SPI, and UART. A detailed specification comparison between the commercially available controllers and the *TinZr* is available in Table 1. Importantly, the *TinZr* can be programmed via the Arduino ecosystem, lowering the barrier to use. This functional density enables multimodal physiological signal acquisition, on-device processing, and flexible data handling (streaming or local logging). All production files for the *TinZr*, including PCB designs and 3D enclosure models for wearables applications, are publicly available at <https://osf.io/hx4cm/>. The onboard microSD reader supports long-duration, high-resolution logging, essential for field or extended experiments where continuous wireless transmission may not be feasible. Wireless connectivity via Wi-Fi and BLE remains available when real-time streaming is needed. We provide an accompanying Arduino library, *TinZrConnect*, to facilitate programming and integration of *TinZr* with existing systems.

Overall, *TinZr* offers a low-cost, power-efficient, and compact alternative that surpasses lighter platforms by combining local storage, wireless flexibility, and standardized sensor interfacing in a single module.

	<i>Seeed XIAO ESP32-C3</i> [20]	<i>DORHEA ESP32-C3 Mini</i> [21]	<i>DFRobot Beetle ESP32-C6</i> [22]	<i>Adafruit GEMMA v2</i> [23]	<i>PJRC Teensy 4.1</i> [24]	<i>TinZr</i>
Dimensions (mm)	21 x 17.8	22 x 18	25 x 20	28 (diameter)	61 x 18	25 x 25
Nbr. GPIOs	11	11	13	3	55	10
Nbr. Analog Pins	4	4	7	1	18	5
Nbr. Digital Pins	11	11	13	3	55	10
I ² C	Yes	Yes	Yes	Yes	Yes	Yes
SPI	Yes	Yes	Yes	Yes	Yes	Yes
UART	Yes	Yes	Yes	No	Yes	Yes
On-board MicroSD Card Reader	No	No	No	No	Yes	Yes
On-board Battery Management	Yes	No	Yes	No	No	Yes
On-board QWIIC Connector	No	No	No	No	No	Yes
On-board IMU	No	No	No	No	No	Yes
On-board Controllable RGB LED	No	No	No	No	Yes	Yes
On-board Push Button	Yes (Reset & Boot)	Yes (Reset & Boot)	Yes (Reset & Boot)	Yes (Reset)	Yes	Yes (Reset & Boot/Control)
Unit Price (in USD) (January, 2026)	~ \$5 – 10	~ \$8 – 12	~ \$7 – 10	~ \$9.95	~ \$30 – 50	~ \$65

Table 1: Features and price comparison between some commercially available controllers and *TinZr* in terms of dimensions, GPIOs, interfaces, and price.

3. Hardware description

Our repository, available at <https://osf.io/hx4cm/>, provides the following materials:

- raw and editable schematics and PCB layout files used in the design of the *TinZr ESP32-C3 Rev5*; PCB layouts are provided in .eprj format for EasyEDA Pro [25], as well as in .kicad_pcb/.kicad_sch format for KiCad [26];

- a 3D model of the wearable enclosure, designed to house the *TinZr* device and battery, provided in .step format and compatible with FreeCAD [27] and other CAD software;
- exported manufacturing files, including Gerber files, pick-and-place data, and bills of materials (BOM) for the PCBs, as well as .stl files for 3D printing the wearable enclosure.

Figure 1 illustrates the *TinZr* pinout along with the top and bottom layers of the PCB. Figure 2 shows the 3D model of the wearable *TinZr* enclosure, illustrating the exploded and assembled views used to house the PCB and battery. The major functional blocks and embedded components of the system are described in the following subsections.

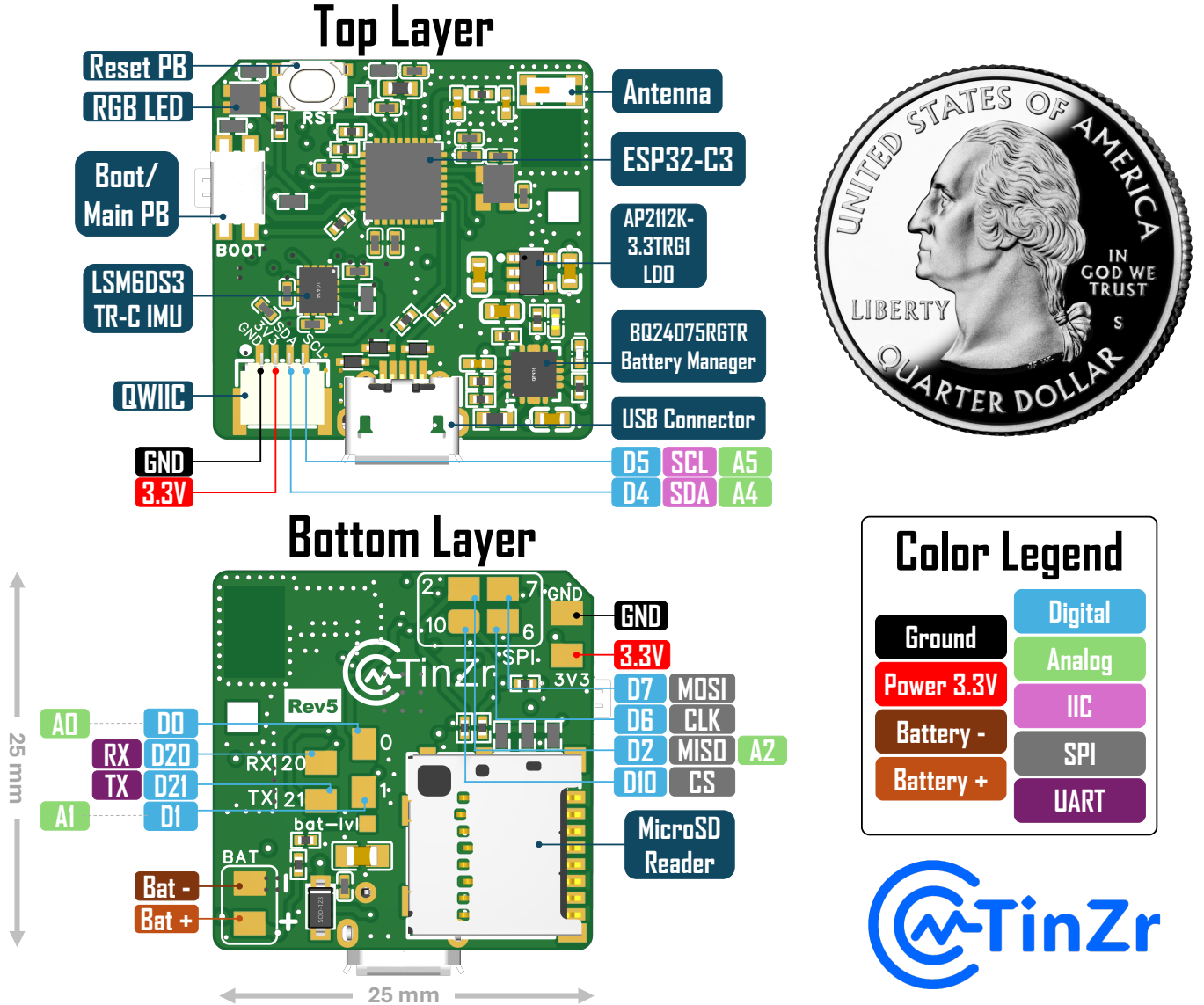


Figure 1: *TinZr* pinout diagram showing the available GPIOs, communication interfaces, power rails, and key functional connectors.

3.1 ESP32-C3 Development Board

The central component of the *TinZr* is an ESP32-C3-based microcontroller module, labeled “ESP32-C3” in Figure 1. The ESP32-C3 module integrates a single-core RISC-V processor with native 2.4 GHz Wi-Fi and

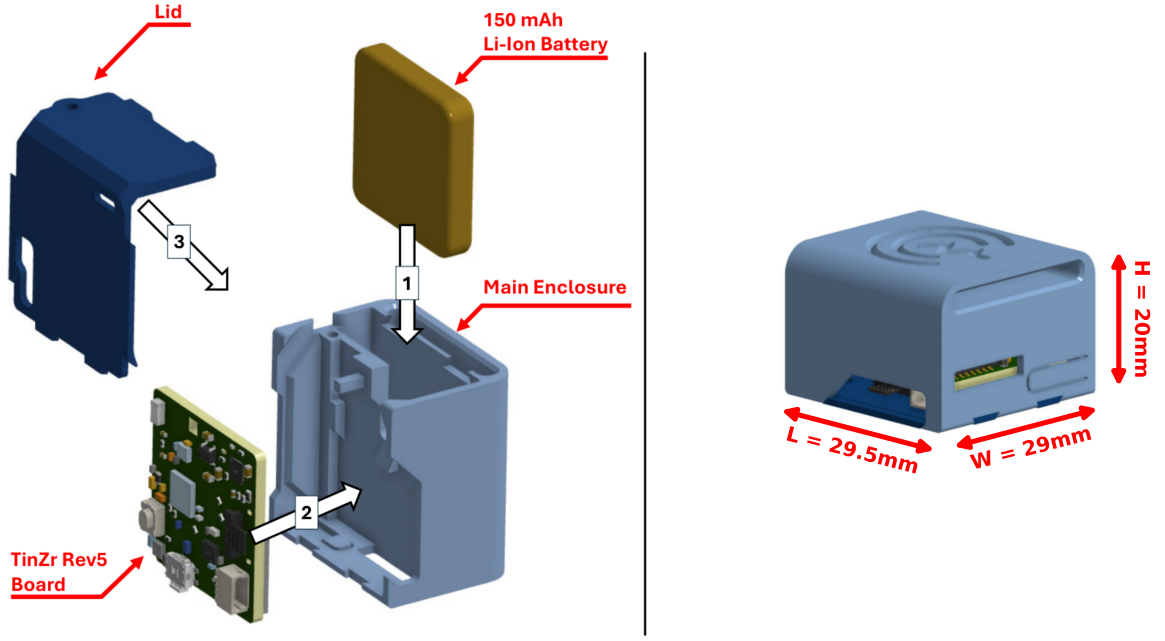


Figure 2: 3D model of the *TinZr* wearable enclosure illustrating the mechanical design used to enclose the PCB and battery for wearable applications. The exploded view (left panel) shows the individual enclosure components, including the main enclosure, top lid, battery lid, *TinZr* Rev 5 PCB, and 150 mAh Li-ion battery, while the assembled view (right panel) shows the fully assembled enclosure (length=29.5mm, width=29mm, and height=20mm). Dashed orange arrows in the exploded view indicate the assembly directions and relative placement of each component during enclosure assembly.

Bluetooth Low Energy (BLE) connectivity, providing a compact and power-efficient solution well suited for wearable and portable applications. The processor operates at clock frequencies of up to 160 MHz, offering sufficient computational performance for real-time control, synchronization, and wireless communication tasks.

The ESP32-C3 module provides a sufficient number of general-purpose input/output (GPIO) pins to support the required digital and analog interfaces, including I²C, SPI, and UART communication buses, while maintaining low power consumption during both active operation and deep sleep modes. Although the ESP32-C3 module offers fewer GPIOs than larger development boards such as the Teensy 4.1, the pin allocation in *TinZr* was carefully designed to prioritize the interfaces and peripherals most relevant to biosignal synchronization, wearable sensing, and wireless data transfer. As a result, all core system functionalities are supported without reliance on external multiplexing or expansion hardware.

The ESP32-C3 module is positioned centrally on the *TinZr* PCB, with its USB interface accessible at the board edge to facilitate firmware development, debugging, and power delivery (Figure 1).

Overall, the ESP32-C3 module provides a compact, wireless, and power-efficient computational core that enables *TinZr*'s functionality while supporting scalable, open-hardware development.

3.2 Inertial Measurement Unit (IMU)

Motion sensing in the *TinZr* platform is provided by an STMicroelectronics LSM6DS3TR-C inertial measurement unit (IMU), labeled "LSM6DS3" in Figure 1. The LSM6DS3TR-C unit integrates a 3-axis accelerometer and a 3-axis gyroscope in a compact, low-power package, enabling continuous monitoring of linear acceleration and angular velocity.

The accelerometer supports selectable full-scale ranges up to ± 16 g, while the gyroscope supports angular

rate measurements up to ± 2000 dps, allowing the device to capture both fine-grained motion and high-dynamic activities. The IMU supports output data rates up to several kilohertz and communicates with the ESP32-C3 module via the I²C bus, providing reliable and low-latency motion data acquisition.

The LSM6DS3TR-C unit was selected for its low power consumption, small footprint, and proven reliability in wearable and embedded applications. In the context of *TinZr*, IMU data can be used to support motion-aware sensing, artifact detection, and synchronization tasks, particularly in mobile and wearable experimental settings.

3.3 Battery Management and Power Regulation

Power management in the *TinZr* platform is handled by a Texas Instruments BQ24075RGTR battery management integrated circuit, labeled “BQ24075” in Figure 1. The BQ24075RGTR unit is a single-cell lithium-ion/lithium-polymer (Li-ion/Li-Po) charger with integrated power-path management, enabling simultaneous battery charging and system operation.

The device supports USB-compatible input sources and provides regulated power to the system while safely managing battery charging, including preconditioning, constant-current, and constant-voltage charging phases. Integrated safety features, such as thermal regulation, overvoltage protection, and charge termination, ensure reliable and safe operation in wearable and portable applications.

Power-path management allows *TinZr* to operate directly from an external power source when available, while automatically switching to battery power when disconnected, without interrupting system operation. This capability is particularly important for long-duration experiments and mobile use cases, where uninterrupted operation and safe battery handling are critical. The BQ24075RGTR unit was selected for its compact footprint, low external component count, and proven reliability in battery-powered embedded systems.

The charging behavior of the BQ24075RGTR unit in *TinZr* is configured using three external resistors connected to the “ISET”, “ILIM”, and “TMR” pins. The fast-charge current is set by a resistor on the “ISET” pin. With an 8.2 k Ω resistor, the resulting charge current is approximately 100 mA, which is well matched to the 150 mAh single-cell lithium-polymer battery [28] used with the *TinZr* and corresponds to a charge rate of approximately 0.7C.

The maximum input current drawn from the external power source is limited via the “ILIM” pin to ensure compatibility with USB power supplies while supporting simultaneous system operation and battery charging. With a 3 k Ω resistor connected to “ILIM”, the input current is limited to approximately 520 mA, providing sufficient headroom for system operation while remaining within typical USB power constraints.

In addition, a resistor connected to the “TMR” pin sets the safety timeout for the charging process. With a 46.4 k Ω resistor, the precharge safety timer is approximately 37 minutes, and the fast-charge safety timeout is approximately 6.2 hours, providing protection against abnormal charging conditions. These resistor values were selected specifically for the 150 mAh battery [28] used with the *TinZr*, and can be adjusted (by desoldering the onboard resistors and replacing them with desired ones) to accommodate larger-capacity batteries by increasing the allowable charge and input currents, while maintaining manufacturer-recommended charge rates and thermal limits.

The battery is soldered directly to the rear side of the PCB at the pads labeled (+) and (-), ensuring correct polarity and compact mechanical assembly.

3.4 On-board Controllable RGB LED

TinZr includes an addressable RGB status LED (“NeoPixel”-class), labeled “RGB LED” in Figure 1. This type of LED integrates red, green, and blue emitters together with a small control driver in a single package, allowing the LED color and brightness to be digitally controlled from a single microcontroller GPIO pin.

The RGB LED can be programmed to provide a visual indication of device state during development and deployment (e.g., BLE advertising/connection state, recording status, or error conditions). Since the LED can draw non-negligible current at high brightness, *TinZr* firmware limits brightness and disables the LED during low-power operation when appropriate. This feature preserves battery life while retaining a simple, user-facing diagnostic interface.

The RGB LED is a fully software-controllable indicator intended for general system feedback and debugging. *TinZr* also includes dedicated power and charging LEDs that operate independently of firmware and provide continuous hardware-level indication of battery and charging status.

3.5 microSD Card Interface

Local data storage in the *TinZr* platform is provided by an on-board microSD card reader, labeled “microSD Reader” in Figure 1. The microSD interface allows sensor data, timestamps, and system logs to be stored directly on removable flash media, enabling fully standalone operation without reliance on a continuous wireless connection.

The microSD card communicates with the ESP32-C3 module via the SPI bus, providing a simple and widely supported interface for embedded systems. This design choice ensures broad compatibility with standard microSD cards and commonly used embedded file systems, while maintaining predictable timing and resource usage. The SPI-based implementation also simplifies integration alongside other peripherals on the shared communication bus.

The inclusion of local storage is particularly important for long-duration recordings, field deployments, and wearable experiments where wireless bandwidth or connectivity may be limited or unavailable. By offloading data logging to the microSD card, *TinZr* supports reliable data capture with minimal impact on wireless performance and power consumption. We validated data logging using a 32 GB microSDHC card (Silicon Power Elite, FAT32-formatted).

3.6 QWIIC Connector

The *TinZr* platform includes an on-board QWIIC-compatible expansion connector, labeled “QWIIC” in Figure 1, providing a standardized interface for external peripheral integration. The QWIIC connector exposes the I²C bus along with power and ground, enabling rapid connection of compatible sensors and modules without soldering.

By leveraging the I²C interface, the QWIIC connector allows additional devices – such as environmental sensors, auxiliary motion sensors, or other low-power peripherals – to be easily integrated into the system. This design supports modular expansion while minimizing wiring complexity and reducing the risk of incorrect connections during prototyping or deployment.

The inclusion of a QWIIC connector enhances the flexibility of the *TinZr* platform by enabling rapid hardware extensions without modifying the core PCB design. This feature is particularly useful for experimental and research-oriented applications, where sensor configurations may evolve over time or vary between deployments.

3.7 Reset and User Push Buttons

TinZr includes two on-board push buttons: a dedicated reset button, labeled “Reset PB”, and a multifunction push button, labeled “Boot/Main PB” in Figure 1. The reset button provides a hardware reset of the ESP32-C3 module, allowing the system to be restarted without cycling power during development or troubleshooting.

The main PB button serves a dual purpose. When used in conjunction with the reset sequence, it can be used to place the ESP32-C3 module into bootloader mode for firmware flashing. During normal operation, the same button can be repurposed by the firmware as a general user input, enabling application-specific functionality such as mode selection, event marking, or user interaction.

The main PB is implemented as right-angle (90°) tactile switch. This orientation allows the button to be actuated from the side of the enclosure rather than the top surface of the PCB, simplifying mechanical integration and improving usability in compact enclosures. The right-angle design is particularly advantageous for wearable applications, where low profile, accessibility, and protection against accidental presses are important considerations.

3.8 Micro-USB Connector

The *TinZr* platform includes a micro-USB connector, labeled “USB Connector” in Figure 1. It provides both power input and a wired communication interface to the ESP32-C3 module. This connector is used for

battery charging via the on-board power management circuitry as well as for firmware flashing, debugging, and development during device configuration.

When connected to an external power source, the micro-USB interface supplies power to the system and enables charging of the on-board lithium-polymer battery through the BQ24075 battery management IC. During development, the same interface allows firmware to be uploaded to the ESP32-C3 module and supports serial communication for debugging and status monitoring.

The micro-USB connector was selected for its compact form factor, mechanical robustness, and wide availability of cables, making it well suited for both early-stage development and enclosed wearable applications. Its placement at the edge of the PCB facilitates straightforward access when the device is integrated into a protective enclosure.

4. Design files summary

Design file-name	File type	License	Location of the file
<i>TinZr</i> Rev 5 PCB	raw schematics, *.svg & *.kicad_sch; PCB sketches, *.kicad_pcb; EasyEDA Pro source, *.eprj; manufacturing files, *.grb & *.csv	CERN OHL-P 2.0	https://osf.io/hx4cm/
Enclosure 3D designs	source, *.step; manufacturing, *.stl	CERN OHL-P 2.0	https://osf.io/hx4cm/
BOM of all components	*.csv	CERN OHL-P 2.0	https://osf.io/hx4cm/
Firmware code	Arduino IDE source, *.ino; Python-based GUIs, *.py	GNU General Public License, version 3 (GPLv3)	https://github.com/ludvikalkhoury/TinZr.git

- *TinZr* Rev5 PCBs: design files of *TinZr* Rev5 PCB are available. These are the raw and editable EasyEDA Pro files (*.eprj format) as well as KiCAD PCB files (.kicad_pcb format). Also available are manufacturing files that can be used directly to produce the PCBs. These are Gerber (.grb), pick and place (.csv), and bill of materials (BOM) (.csv) files.
- Enclosure 3D designs: raw editable 3D design files (in .step format) are available for the *TinZr*'s wearable enclosure. Also are available the corresponding .stl files that can be used directly in slicing software to generate a g-code for a 3D printer.
- BOM of all components: a BOM is available that contains all the components we used to build the *TinZr* Rev5 PCB.
- Firmware code: an Arduino library, *TinZrConnect*, is available, with examples that can be directly updated and run on the *TinZr*.

5. Bill of materials (BOM)

We provide a detailed and comprehensive BOM in the *TinZr*'s OSF repository <https://osf.io/hx4cm/>, listing all electronic components along with their part numbers, manufacturers, and supplier IDs.

The cost of making the *TinZr* Rev5 is around \$65 per unit. This per-unit cost assumes a minimum quantity of 5, reflecting the minimum quantity required by the PCB manufacturing company, and include shipping and current customs duties (December 2025).

6. Build instructions

Step-by-step instructions are provided, on how to manufacture the *TinZr*.

6.1 PCB Manufacturing

As previously mentioned, all the files needed in the PCB manufacturing process are publicly available. These files are Gerber, Pick-and-Place, and BOM, and they can be found in the OSF repository, <https://osf.io/hx4cm/>. We have used JLCPCB (<https://jlcpcb.com/>)¹ to manufacture the boards since it provides a wide variety of base materials and colors and other PCB specifications. We designed and built the *TinZr* Rev 5 as a 4-layer PCB, using a standard quality PCB with flame retardant (FR-4) as the base material. We picked a PCB thickness of 1.6 mm with a hot air solder leveling (HASL) (with lead) surface finish. We also chose the copper weight on the top and bottom to be 1 oz/ft². We used two ground planes (top and bottom).

6.2 Wearable Enclosure 3D Printing

We provide a 3D model (.stl files) for an enclosure designed to fit the *TinZr* Rev5's PCB as well as a 150 mAh battery that power the *TinZr*.

The wearable enclosure was printed using a local Creality Ender-3 3D printer (Fused Deposition Modeling (FDM)), using PLA filaments. The quality of the print was acceptable for our purposes. In case the user requires a higher resolution, we suggest using a higher quality alternatives such as Multi-Jet Fusion (MJF) printing, which could result in a higher-quality product (but comes at a higher price).

6.3 Component Soldering

For our design, we used the mounting and soldering services provided by JLCPCB to assemble and solder all components on the *TinZr* board. The only exception was the main push button, which was not available through JLCPCB at the time of ordering and was therefore soldered manually.

6.4 Assembly

The *TinZr* Rev 5 is fully functional out-of-the-box and does not require any assembly for basic operation. However, when integrating the system into a wearable enclosure, the following steps should be followed to properly assemble the wearable unit.

1. Solder the battery positive (+) and negative (−) leads to the corresponding battery terminals located on the bottom side of the *TinZr* board (Figure 3, Step 1).
2. Insert the battery into the designated battery compartment and place the *TinZr* board onto its designated mounting platform within the enclosure (Figure 3, Step 2).
3. Clip the snap-fit lid onto the main enclosure to secure the assembly (Figure 3, Step 3). The wearable unit is now fully assembled and ready for use.

The final panel of Figure 3, labeled Full Assembly, shows the completed *TinZr* Rev 5 wearable unit after all assembly steps have been performed.

TinZr provides a battery-level monitoring feature. Battery voltage is measured via the *bat-lvl* pad, which must be connected by soldering to the Pin 1 pad (see the bottom layer of Figure 1); this pin is designated in the *TinZr* firmware as the default battery-sensing input. This connection is required for the battery level to be reported correctly in the example firmware included in the repository.

¹Other PCB manufacturers offer comparable services – for example, DKRed (<https://www.digikey.com/en/resources/dkred>), OSH Park (<https://oshpark.com/>), and PCB Way (<https://www.pcbway.com/>), to name a few.

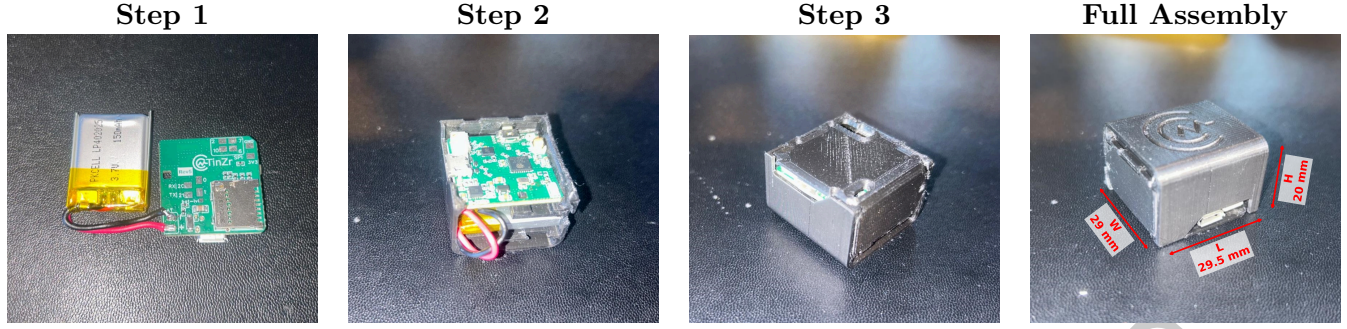


Figure 3: Step-by-step illustration on how to assemble the *TinZr* Rev 5 board with the wearable 3D enclosure, following the steps described in Section 6.4. The last panel shows the fully assembled device (length=29.5mm, width=29mm, and height=20mm).

6.5 Programming

The *TinZr* Arduino library, *TinZrConnect*, can be downloaded directly from the *TinZr* GitHub repository <https://github.com/ludvikalkhoury/TinZr.git>. In addition to the Arduino library, *TinZr* provides a custom Arduino Board Manager package with full support for the *TinZr* Rev 5 hardware, enabling seamless installation and configuration within the Arduino IDE. The *TinZrConnect* library includes a collection of example sketches designed to test and demonstrate the full functionality of the *TinZr* Rev 5 board.

For instance, the *TinZrCore* sketch provides a basic demonstration of core hardware features, including the main push button, status LED, and battery level monitoring (assuming the battery-level pad has been soldered to Pin 1 pad as described in Section 6.4).

The *TinZrComCenter* sketch establishes a centralized communication framework in which multiple *TinZr* devices connect to a single host system acting as a communication hub. In this architecture, each *TinZr* operates as an independent node that can transmit data and receive control messages over Wi-Fi. The central hub manages all connected devices and can selectively broadcast commands to all *TinZr* units or address individual devices independently. This design enables coordinated multi-device operation and is well suited for distributed or swarm-style applications, such as synchronized sensing or collaborative robotics.

Another example, *TinZrWearableSD*, enables communication between one or more *TinZr* devices and a central PC over Bluetooth Low Energy (BLE), allowing the PC to control recording start and stop events while enabling local logging of inertial measurement unit (IMU) data (accelerometer and gyroscope) as well as red and infrared photoplethysmographic (PPG) data when a MAX30102 sensor [29] is connected via QWIIC.

Both *TinZrComCenter* and *TinZrWearableSD* are accompanied by graphical user interfaces (GUIs), along with the underlying Python source code, allowing users to operate the system without requiring extensive experience in firmware or software development. In later sections of this paper, we demonstrate the capabilities of the *TinZr* Rev 5 using *TinZrComCenter* and *TinZrWearable* as representative example use cases.

7. Operation instructions

7.1 User Instructions

The *TinZr* Rev 5 operates similarly to other microcontrollers in the ESP32 family. Here we follow a simple three-step workflow: **Power**, **Program**, and **Deploy**.

1. **Power:** The *TinZr* Rev 5 can be powered either by connecting it to a host computer via USB or by using a connected rechargeable battery for standalone or wearable operation. When powered through USB, a white status LED illuminates, indicating that the board is receiving power. When a battery is connected and charging, a green charging-status LED remains illuminated and turns off once charging is complete. When operating solely on battery power, no LEDs are illuminated. The power and charging status LEDs are located to the left of the USB connector at the same vertical level (see Figure 1, top layer).

2. **Program:** The *TinZr* Rev 5 can be programmed with any ESP32-compatible firmware, including user-developed sketches or example programs provided with the *TinZrConnect* library. Programming is performed using the Arduino IDE after selecting the appropriate board and serial port. In some cases, the board must be placed into bootloader mode manually. This is accomplished by pressing and holding the main button, briefly pressing and releasing the reset button, and then releasing the main button. Once the firmware upload is complete and the program begins execution, the device is ready for use.
3. **Deploy:** After programming, the *TinZr* Rev 5 can be deployed either while connected via USB or as a fully standalone device powered by the onboard battery. In deployed operation, the device can stream sensor data, communicate wirelessly using BLE or Wi-Fi, or operate autonomously depending on the loaded firmware. This flexibility allows the *TinZr* to function both as a development platform and as an embedded wearable system.

7.2 Safety Considerations

The *TinZr* Rev 5 is a low-voltage, battery- and USB-powered microcontroller platform intended for embedded sensing, wearable prototyping, and wireless data acquisition. The device does not make direct electrical contact with the wearer, nor does it generate or apply electrical stimulation or excitation signals. As such, when used as intended, the *TinZr* presents minimal inherent electrical risk.

When deployed in wearable or human-adjacent applications, the *TinZr* is typically powered solely by its onboard battery and communicates wirelessly via BLE or Wi-Fi. This battery-only, wireless mode of operation significantly reduces the risk of electrical shock associated with mains-powered systems.

If the *TinZr* is connected to external sensors or peripherals that may contact a human subject, it is the user's responsibility to ensure that those components meet appropriate electrical safety and isolation standards. The *TinZr* Rev5 has not been tested, certified, or approved for clinical or medical use, and should therefore be used exclusively for research, prototyping, or non-clinical applications unless additional safety certification is obtained.

As with most open-source hardware platforms, the *TinZr* is provided without warranty, and its use is undertaken at the user's own risk. The authors disclaim all liability to the extent permitted by applicable law.

8. Validation and characterization

To validate the operation and characterize the performance of the *TinZr* Rev 5 platform, we conducted two complementary test cases. These were designed to evaluate the system across key intended use scenarios, namely, (a) multi-device communication and coordination and (b) real-time sensor data acquisition.

8.1 Multi-Device Communication and Coordination

Multi-device communication and coordination were validated using the *TinZrComCenter* framework. In this configuration, multiple *TinZr* devices simultaneously connect to a central host system (a personal computer) that functions as a communication hub. The firmware supports bi-directional communication over TCP/IP, enabling both TinZr-to-hub and hub-to-TinZr messaging, as well as broadcast commands addressed to all connected *TinZr* units. This test case demonstrates the *TinZr* platform's ability to support coordinated, multi-node operation, enabling use cases such as distributed sensing, synchronized data acquisition, and swarm-style control architectures. For demonstration purposes, we deployed the Arduino-based firmware located at:

```
TinZr/libraries/TinZrConnect/examples/TinZrComCenter/TinZrComCenter.ino
```

First, the user should connect the PC to the same Wi-Fi network to which the *TinZrs* are intended to connect. In the firmware, the user should provide the Wi-Fi SSID and password, the PC's IP address, and a hostname for the *TinZr* onto which the code is flashed. Here, we upload the firmware to two *TinZr* devices, referred to as

TinZrBlack (black-colored enclosure) and *TinZrGold* (gold-colored enclosure). Next, we launch the companion Python-based graphical user interface (GUI)² located at:

`TinZr/libraries/TinZrConnect/examples/TinZrComCenter/TinZrComCenter.py`

After both devices successfully connected to the hub, a broadcast command was issued to set the onboard RGB LEDs to blue by sending the command **LED 0 0 255 20**, which specifies red, green, and blue color intensities of 0, 0, and 255, respectively, with a low brightness level of 20. As shown in the left panel of Figure 4, the RGB LEDs on both *TinZr* devices simultaneously turned blue, confirming correct broadcast communication and synchronized command execution. Additionally, we can increase the brightness level to 150 and LEDs on both *TinZr* boards get brighter (right panel of Figure 4).

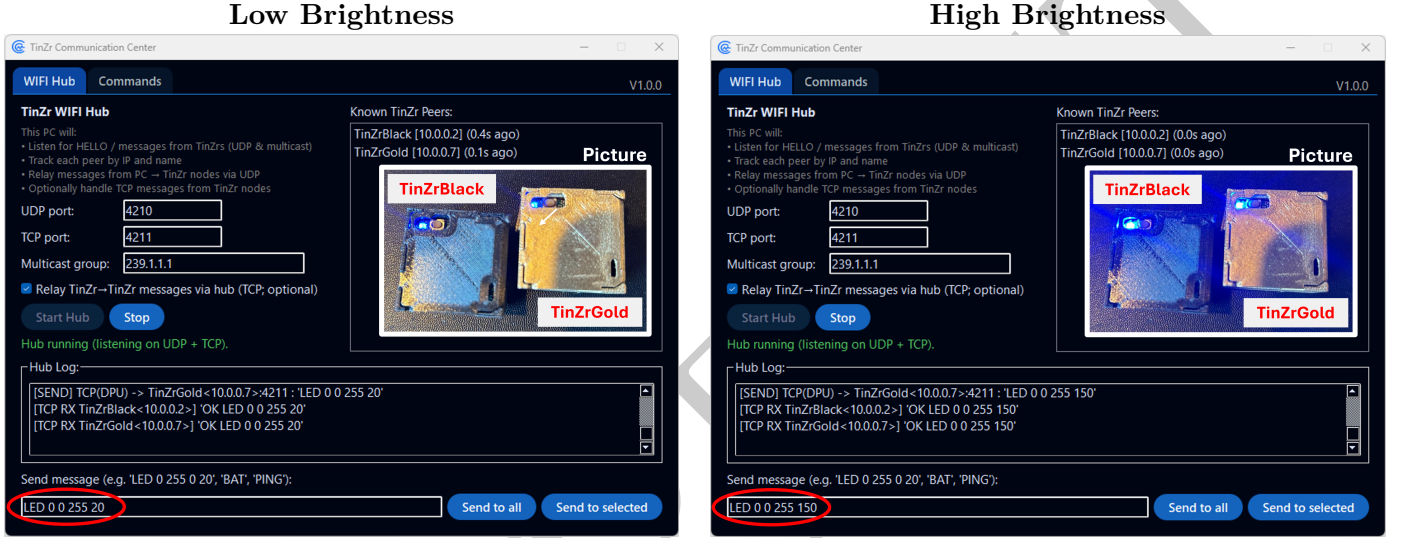


Figure 4: Screenshot of the *TinZr* Communication Center Python-based GUI used for multi-device control over Wi-Fi, alongside a photograph of two *TinZr* devices (*TinZrBlack* and *TinZrGold*). The left panel demonstrates LED control at a low brightness setting (brightness level 20), while the right panel shows the same devices driven at a high brightness setting (brightness level 150).

8.2 Real-Time Sensor Data Acquisition

We also demonstrated end-to-end wirelessly-controlled acquisition of inertial data using the *TinZrWearableSD* configuration. In this mode, the *TinZr* device streams inertial measurement unit (IMU) data acquired from the LSM6DS3TR-C sensor, including triaxial accelerometer and gyroscope signals³. Control of recording operations, including start and stop commands, is handled through an accompanying Python-based graphical user interface (GUI) that communicates with the *TinZr* devices over BLE, while all sensor data are logged locally to the onboard microSD card. For demonstration purposes, we deployed the wearable firmware located at:

`TinZr/libraries/TinZrConnect/examples/TinZrWearableSD/TinZrWearableSD.ino`

The user should provide a hostname for the *TinZr* on which the code is flashed. After the firmware is flashed onto the board, we launch the companion Python-based GUI⁴ located at:

²We also provide an executable file that installs the GUI on the PC without the need to run any Python codes. This file is available in the repository under the *TinZrComCenter* directory and as part of the repository release.

³This firmware also supports the addition of a photoplethysmographic (PPG) sensor, MAX30102, when the optical module is present. The MAX30102 interfaces with the *TinZr* board via an I²C connection using the onboard QWIIC connector. When no MAX30102 sensor is connected, the corresponding PPG channels are reported as zero-valued signals.

⁴Similar to the previous example, we also provide an executable file that installs the GUI on the PC without the need to run any Python codes. This file is available in the repository under the *TinZrWearableSD* directory, and as part of the repository release.

For illustration purposes, we collected IMU data under three conditions and report representative snapshots of the streamed sensor data in Figure 5.

In the first condition, data were collected while the participant sat still while holding the *TinZr* in the right hand (data reported in the left panel of Figure 5). Next, the data were then collected during repetitive hand up-and-down motion at a frequency of 50 bpm (beats per minute) (middle panel of Figure 5). Finally, the motion was repeated at a frequency of 100 bpm (right panel of Figure 5). The hand-motion frequency was controlled using a metronome.

In the absence of motion, the IMU signals show minimal activity. When motion is introduced, the IMU signals capture the corresponding periodic movement (middle panel of Figure 5). This effect becomes more frequent, with a higher rate of oscillations, as the motion frequency increases from 50 bpm to 100 bpm (right panel of Figure 5).

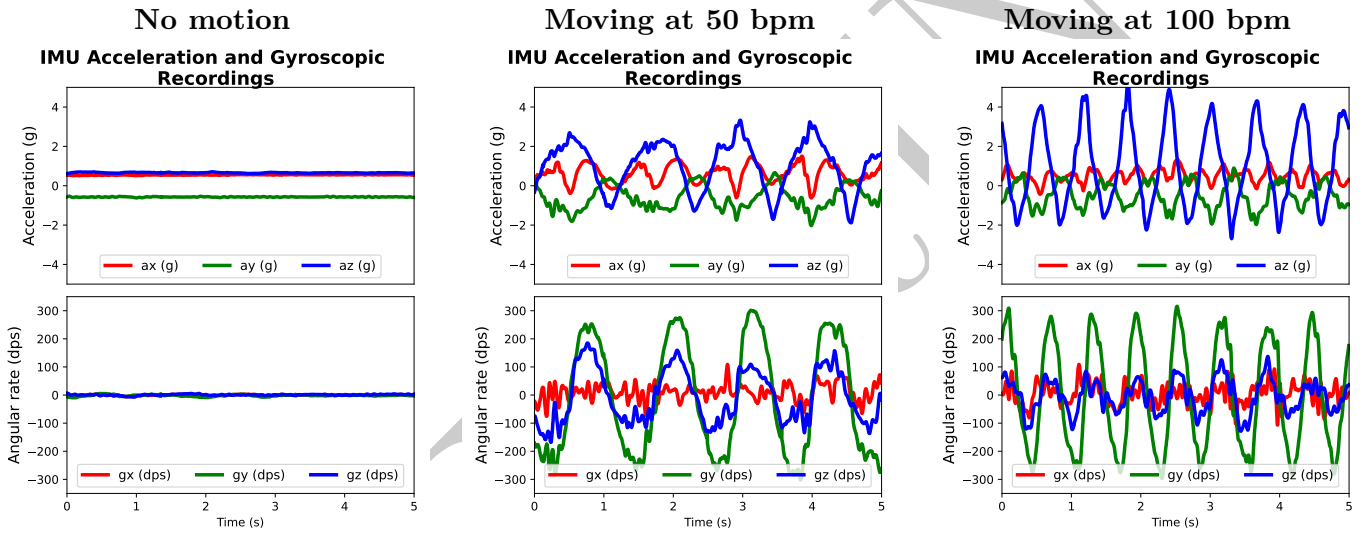


Figure 5: IMU recordings during three scenarios; left panel, under no motion; middle panel, during repetitive hand up-and-down motion at a frequency of 50 bpm; right panel: during repetitive hand up-and-down motion at a frequency of 100 bpm.

Ethics statement

The data collection from the participant reported in Section 8.2 was approved by Weill Cornell Medicine Institutional Review Board (WCM-IRB) (IRB number 19-05020070). The study was conducted in accordance with the local legislation and institutional requirements. Written informed consent was obtained by the participants' legal guardians/next of kin.

9. CRediT author statement

Ludvik Alkhoury: Conceptualization, Methodology, Software, Validation, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing. **Tony Moore:** Conceptualization, Resources, Visualization, Writing - Review & Editing. **Petrus Swissler:** Resources, Visualization, Writing - Review & Editing. **N. Jeremy Hill:** Resources, Visualization, Writing - Review & Editing. **Sudhin A. Shah:** Resources, Visualization, Writing - Review & Editing. **Moshe Kam:** Resources, Visualization, Funding Acquisition, Supervision, Writing - Review & Editing.

10. Declaration of competing interest

Ludvik Alkhoury, N. Jeremy Hill, and Sudhin A. Shah hold equity in and serve as the founding team for Cognitive Signals Inc. These interests have been disclosed to the authors' respective institutions and are managed in accordance with their conflict-of-interest policies. The remaining authors declare no competing interests.

11. Acknowledgements

This work was funded by the New Jersey Institute of Technology (NJIT), Office of the Dean, Newark College of Engineering (NCE). Author N. Jeremy Hill acknowledges support from the NIH (grant P41EB018783), from NY State SCIRB (C37714GG-IDEA and C41212GM-IDEA), and from the Stratton VA Medical Center.

References

- [1] Jayraj V. Vaghasiya, Carmen C. Mayorga-Martinez, and Martin Pumera. Wearable sensors for telehealth based on emerging materials and nanoarchitectonics. *npj Flexible Electronics*, 7(1):26, 2023.
- [2] Straits Research. Wearable sensors market size and outlook, 2025–2033. <https://straitsresearch.com/report/wearable-sensors-market>, 2024. Accessed: 2025-10-12.
- [3] Bradley J. Petek, Mostafa A. Al-Alusi, Nathaniel Moulson, Aubrey J. Grant, Cyril Besson, J. Sawalla Guseh, Meagan M. Wasfy, Vincent Gremeaux, Timothy W. Churchill, and Aaron L. Baggish. Consumer wearable health and fitness technology in cardiovascular medicine: JACC state-of-the-art review. *Journal of the American College of Cardiology*, 82(3):245–264, 2023.
- [4] Dhruv R. Seshadri, Ryan T. Li, James E. Voos, James R. Rowbottom, Celeste M. Alfes, Christian A. Zorman, and Colin K. Drummond. Wearable sensors for monitoring the internal and external workload of the athlete. *npj Digital Medicine*, 2(1):71, 2019.
- [5] Rory A. Cooper, George Smolinski, Jorge L. Candiotti, Shantanu Satpute, Garrett G. Grindle, Tawnee L. Sparling, Michelle J. Nordstrom, Xiaoning Yuan, Allison Symsack, Chang Dae Lee, et al. Current state, needs, and opportunities for wearable robots in military medical rehabilitation and force protection. In *Actuators*, volume 13, page 236. MDPI, 2024.
- [6] Patrik Kutilek, Petr Volf, Slavka Viteckova, Pavel Smrcka, Vaclav Krivanek, Lenka Lhotska, Karel Hana, Radek Daskocil, Leos Navratil, Zdenek Hon, et al. Wearable systems for monitoring the health condition of soldiers: Review and application. In *Proceedings of the International Conference on Military Technologies (ICMT)*, pages 748–752. IEEE, 2017.
- [7] Che-Chang Yang and Yeh-Liang Hsu. A review of accelerometry-based wearable motion detectors for physical activity monitoring. *Sensors*, 10(8):7772–7788, 2010.
- [8] Ludvik Alkhoury, JiWon Choi, Vishnu D. Chandran, Gabriela B. de Carvalho, Saikat Pal, and Moshe Kam. Dual-wavelength photoplethysmography framework for heart rate calculation. *Sensors*, 22(24):9955, 2022.
- [9] Ludvik Alkhoury, Ji-Won Choi, Chizhong Wang, Arjun Rajasekar, Sayandeep Acharya, Sean Mahoney, Barry S. Shender, Leonid Hrebien, and Moshe Kam. Heart-rate tuned comb filters for processing photoplethysmogram (PPG) signals in pulse oximetry. *Journal of Clinical Monitoring and Computing*, 35(4):797–813, 2021.
- [10] Mohamed Adel Serhani, Hadeel T. El Kassabi, Heba Ismail, and Alramzana Nujum Navaz. ECG monitoring systems: Review, architecture, processes, and key challenges. *Sensors*, 20(6):1796, 2020.

- [11] Burcu Arman Kuzubasoglu, Ersin Sayar, Cedric Cochrane, Vladan Koncar, and Senem Kursun Bahadir. Wearable temperature sensor for human body temperature detection. *Journal of Materials Science: Materials in Electronics*, 32(4):4784–4797, 2021.
- [12] Ziao Xue, YanSong Gai, Yuxiang Wu, Zhuo Liu, and Zhou Li. Wearable mechanical and electrochemical sensors for real-time health monitoring. *Communications Materials*, 5(1):211, 2024.
- [13] Rahul K. Kher and Dipak M. Patel. A comprehensive review on wearable health monitoring systems. *The Open Biomedical Engineering Journal*, 15(1), 2021.
- [14] Louis J. Dankovich, Janell S. Joyner, William He, Ahmad Sesay, and Monifa Vaughn-Cooke. CogWatch: An open-source platform to monitor physiological indicators for cognitive workload and stress. *HardwareX*, 19, 2024.
- [15] Adafruit Industries LLC. Adafruit QT Py ESP32-S2 WiFi dev board (product id: 5325). <https://www.adafruit.com/product/5325>, 2022. Accessed: 2025-10-13.
- [16] Tommaso Polonelli, Lukas Schulthess, Philipp Mayer, Michele Magno, and Luca Benini. H-Watch: An open, connected platform for AI-enhanced COVID-19 symptom monitoring and contact tracing. In *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE, 2021.
- [17] Sebastian Frey, Giusy Spacone, Andrea Cossettini, Marco Guermandi, Philipp Schilk, Luca Benini, and Victor Kartsch. BioGAP-Ultra: A modular edge-AI platform for wearable multimodal biosignal acquisition and processing. *arXiv preprint arXiv:2508.13728*, 2025.
- [18] Gabriel P. M. Pinheiro, Ricardo K. Miranda, Bruno J. G. Praciano, Giovanni A. Santos, Fábio L. L. Mendonça, Elnaz Javidi, João Paulo da Costa, and Jr. de Sousa, Rafael T. Multi-sensor wearable health device framework for real-time monitoring of elderly patients using a mobile application and high-resolution parameter estimation. *Frontiers in Human Neuroscience*, 15:750591, 2022.
- [19] Texas Instruments. CC2640R2F SimpleLink Bluetooth Low Energy wireless MCU. <https://www.ti.com/product/CC2640R2F>, 2020. Accessed: 2025-10-13.
- [20] Seeed Studio. Getting started with Seeed Studio XIAO ESP32C3. https://wiki.seeedstudio.com/XIAO_ESP32C3_Getting_Started/, 2024. Accessed: 2025-10-13.
- [21] Amazon. ESP32-C3 mini development board. <https://www.amazon.com/ESP32-C3-Development-Bluetooth-Single-Core-Processor/dp/B0D5XNBPBC?th=1>, 2025. Accessed: 2025-10-13.
- [22] DFRobot. Beetle ESP32-C6 mini development board. <https://www.dfrobot.com/product-2778.html>, 2025. Accessed: 2025-10-13.
- [23] Adafruit Industries LLC. GEMMA v2 — miniature wearable electronic platform (product id: 1222). <https://www.adafruit.com/product/1222>, 2025. Accessed: 2025-10-13.
- [24] PJRC. Teensy 4.1 development board. <https://www.pjrc.com/store/teensy41.html>, 2025. Accessed: 2025-10-13.
- [25] EasyEDA Team. EasyEDA: Online PCB design tool. <https://easyeda.com/>, 2024. Accessed: 2024-08-13.
- [26] KiCad Development Team. Kicad EDA. <https://www.kicad.org/>, 2024. Accessed: 2024-08-15.
- [27] FreeCAD Community. FreeCAD: Your own 3d parametric modeler. <https://www.freecad.org/>, 2024. Accessed: 2024-08-15.
- [28] Adafruit Industries LLC. Lithium ion polymer battery — 3.7V 150mAh (product id: 1317). <https://www.adafruit.com/product/1317>. Accessed: 2026-01-12.

- [29] AEDIKO. MAX30102 heart rate sensor module (2-pack). <https://www.amazon.com/AEDIKO-MAX30102-Detection-Concentration-Compatible/dp/B09LQDW27N>, 2026. Accessed: 2026-01-12.

PREPRINT